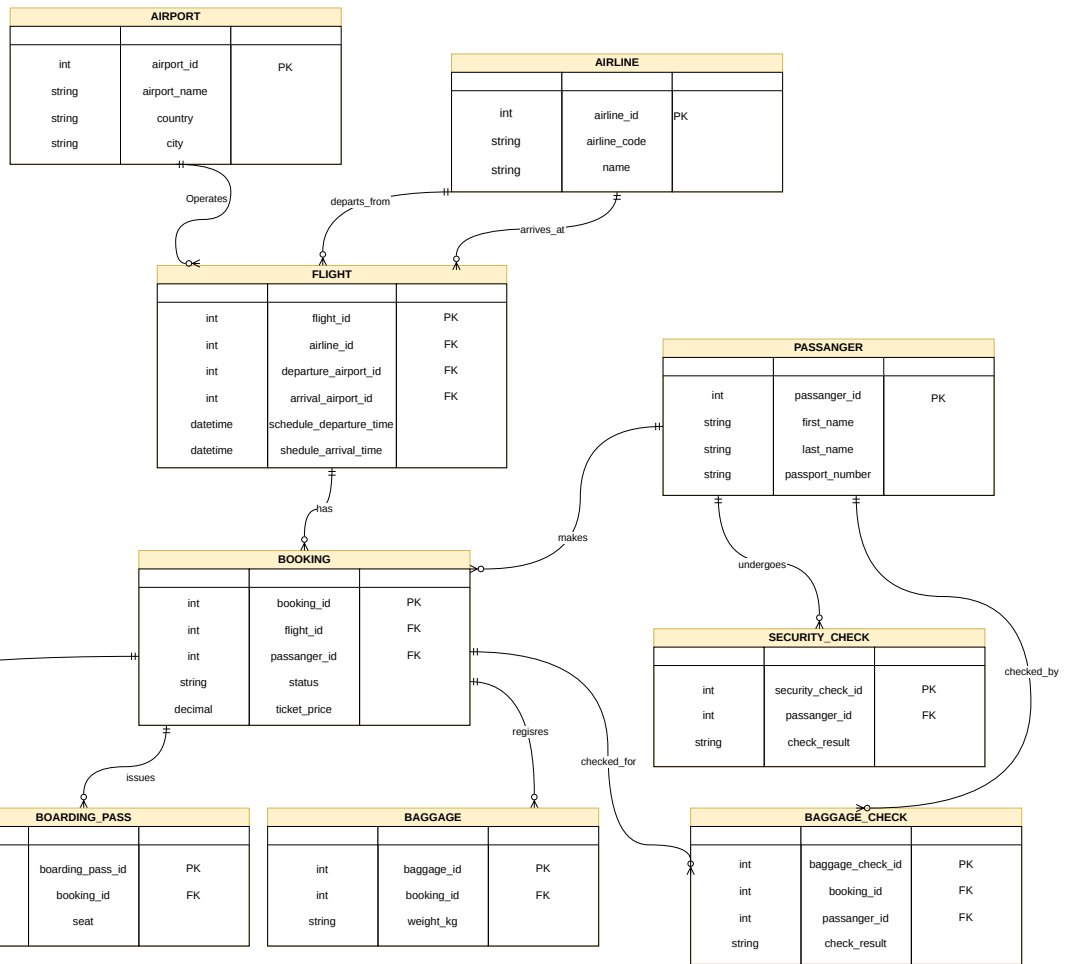


PK - PRIMARY KEY
FK - FOREIGN KEY



Textual Description of the ERD Diagram

Entities and their attributes

In my database I have 10 main entities.

AIRPORT — stores information about airports.

airport_id — INT, PK

airport_name — STRING

country — STRING

city — STRING

AIRLINE — stores information about airlines.

airline_id — INT, PK

airline_code — STRING

name — STRING

FLIGHT — stores information about flights.

flight_id — INT, PK

airline_id — INT, FK

departure_airport_id — INT, FK

arrival_airport_id — INT, FK

schedule_departure_time — DATETIME

shedule_arrival_time — DATETIME

BOOKING — stores information about flight bookings.

booking_id — INT, PK

flight_id — INT, FK

passanger_id — INT, FK

status — STRING

ticket_price — DECIMAL

BOOKING_CHANGE — stores information about changes in bookings.

change_id — INT, PK

booking_id — INT, FK

change_details — STRING

BOARDING_PASS — stores information about boarding passes.

boarding_pass_id — INT, PK

booking_id — INT, FK

seat — STRING

BAGGAGE — stores information about passenger baggage.

baggage_id — INT, PK

booking_id — INT, FK

weight_kg — INT

PASSANGER — stores basic passenger information.

passanger_id — INT, PK

first_name — STRING

last_name — STRING

passport_number — STRING

SECURITY_CHECK — stores the results of passenger security checks.

security_check_id — INT, PK

passanger_id — INT, FK

check_result — STRING

BAGGAGE_CHECK — stores the results of baggage checks.

baggage_check_id — INT, PK

booking_id — INT, FK

passanger_id — INT, FK

check_result — STRING

PK means Primary Key, which is used to identify each record. FK means Foreign Key, which connects one table with another.

Relationships between entities

AIRLINE — FLIGHT:

One airline can have many flights, but each flight belongs to one airline. This is 1:N.

AIRPORT — FLIGHT:

One airport can be used by many flights. A flight has one departure airport and one arrival airport. Both relationships are 1:N.

FLIGHT — BOOKING:

One flight can have many bookings, but each booking belongs to one flight. This is 1:N.

PASSANGER — BOOKING:

One passenger can make many bookings, but each booking belongs to one passenger. This is 1:N.

BOOKING — BOOKING_CHANGE:

One booking can have many changes. Each change belongs to one booking. This is 1:N.

BOOKING — BOARDING_PASS:

A boarding pass belongs to one booking. In my diagram this relationship is 1:1.

BOOKING — BAGGAGE:

One booking can have several baggage records. Each baggage record belongs to one booking. This is 1:N.

PASSANGER — SECURITY_CHECK:

One passenger can have several security checks. Each check belongs to one passenger. This is 1:N.

BOOKING — BAGGAGE_CHECK:

One booking can have several baggage checks. Each baggage check is connected to one booking. This is 1:N.

PASSANGER — BAGGAGE_CHECK:

One passenger can be connected to several baggage checks. Each baggage check is connected to one passenger. This is 1:N.

Business logic

The main idea of my database is to store information about an international airport.

An airline operates flights. Each flight has a departure airport and an arrival airport. Passengers can book these flights.

When a passenger makes a booking, the system stores the booking information, such as its status and ticket price. The booking is connected to both the passenger and the flight.

If something changes in a booking, the change is saved in the BOOKING_CHANGE table. This allows the system to keep information about previous changes.

Passengers can also have baggage and boarding passes. The baggage can be checked, and the result of the check is stored in BAGGAGE_CHECK.

Passengers also pass security checks. The result of each security check is stored in SECURITY_CHECK.

Primary keys are used to identify records, and foreign keys are used to connect the tables together.